IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

P11 – Initiation à la programmation

-

Les structures conditionnelles (ou alternatives)

Hervé L'helguen

Plan

1. Introduction

Introduction

- **Mon problème :** Un marchand d'eau minérale livre ses clients strasbourgeois à vélo électrique et voudrait pouvoir afficher un ticket qui récapitule le montant à facturer après avoir saisi le nom du client et la quantité de bouteilles sachant que le prix de la bouteille est de 57 centimes. Le prix de la livraison évolue et se calcule désormais de la façon suivante :
  - Si le montant est supérieur à 50 € la livraison est gratuite
  - Si le montant est inférieur à 50 € la livraison est facturée 15 % de la commande avec un minimum forfaitaire de 5 €

Pour rappel : `montant = PRIX_UNITAIRE * nbBouteilles`

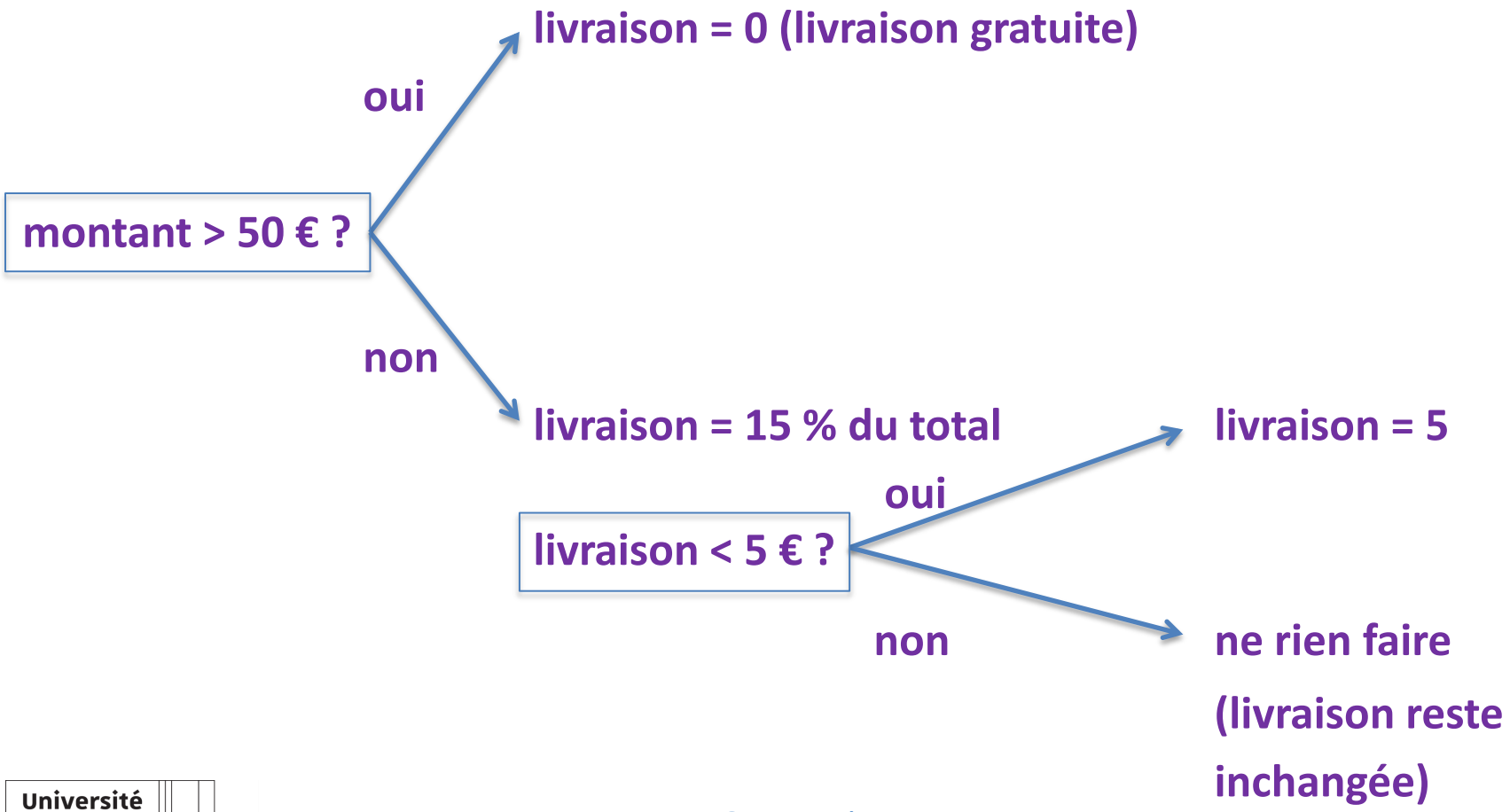
Par exemple :

- montant = 68,40 € → **livraison gratuite**
- montant = 39,90 € → livraison = 15 % de 39,90 soit **5,98 €**
- montant = 13,68 € → livraison = 15 % de 13,68 soit 2,05 €

  dans ce cas on facture de façon forfaitaire **5 €**

Introduction

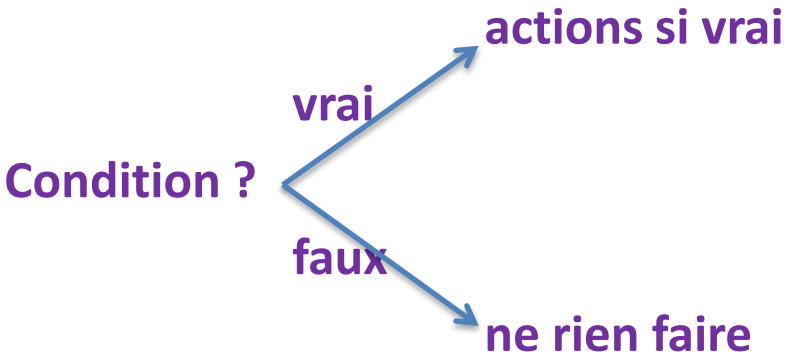
On peut représenter cette situation sous la forme d'un arbre :



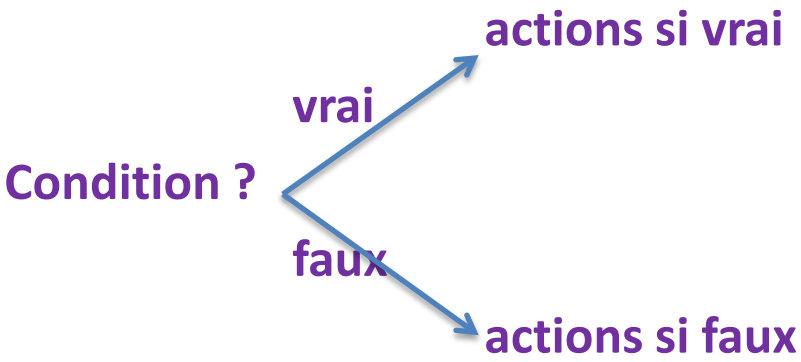
Plan

1. Introduction
2. Structures alternatives dans un algorithme

Structures alternatives dans un algorithme




```
Si condition Alors
    actions si vrai
FinSi
```




```
Si condition Alors
    actions si vrai
Sinon
    actions si faux
FinSi
```

Structures alternatives dans un algorithme

Pour effectuer les conditions il faut introduire les **opérateurs de comparaison** :

==	égal	Si **a == b** Alors
<	strictement inférieur	Si **a < b** Alors
>	strictement supérieur	Si **a > b** Alors
<=	inférieur ou égal	Si **a <= b** Alors
>=	supérieur ou égal	Si **a >= b** Alors
<>	différent	Si **a <> b** Alors

Attention : ne pas confondre l'opérateur d'affection = et l'opérateur de comparaison ==

Structures alternatives dans un algorithme

Lors d'un test, la condition ne peut être évaluée qu'à 2 valeurs **vrai** ou **faux**. Le résultat de la condition est donc de **type booléen** (cf algèbre de Boole). Pour cumuler plusieurs conditions, il faut compléter nos opérateurs avec les **opérateurs booléens** :

NON	Si **NON fini** Alors //idem Si fini == faux Alors
ET	Si **age < 10 ET dep == 67** Alors
OU	Si **dep == 67 OU dep == 68** Alors

Attention aux priorités : NON est prioritaire sur le ET qui est prioritaire sur le OU, dans le doute utilisez des parenthèses.

Déclarations dans un algorithme

Voici 3 tableaux qu'il faut connaître :

a	NON a
vrai	**faux**
faux	**vrai**

a	b	a ET b
faux	faux	**faux**
faux	vrai	**faux**
vrai	faux	**faux**
vrai	vrai	**vrai**

a	b	a OU b
faux	faux	**faux**
faux	vrai	**vrai**
vrai	faux	**vrai**
vrai	vrai	**vrai**

Plan

1. Introduction
2. Structures alternatives dans un algorithme
3. Mettre en place les tests dans notre exemple

Mettre en place les tests dans notre exemple

Comment mettre en place les tests à effectuer ?

```
Algorithme EauxMinerales
    Déclarations
        Constantes
            PRIX_UNITAIRE = 0.57 : réel
            TAUX_LIVRAISON = 0.15 : réel
            MONTANT_MIN = 50 : réel
            MIN_FORFAIT = 5 : réel
        Variables
            nomClient : chaîne de caractères
            nbBouteilles : entier
            montant, livraison, total : réels
 Début
        …
```

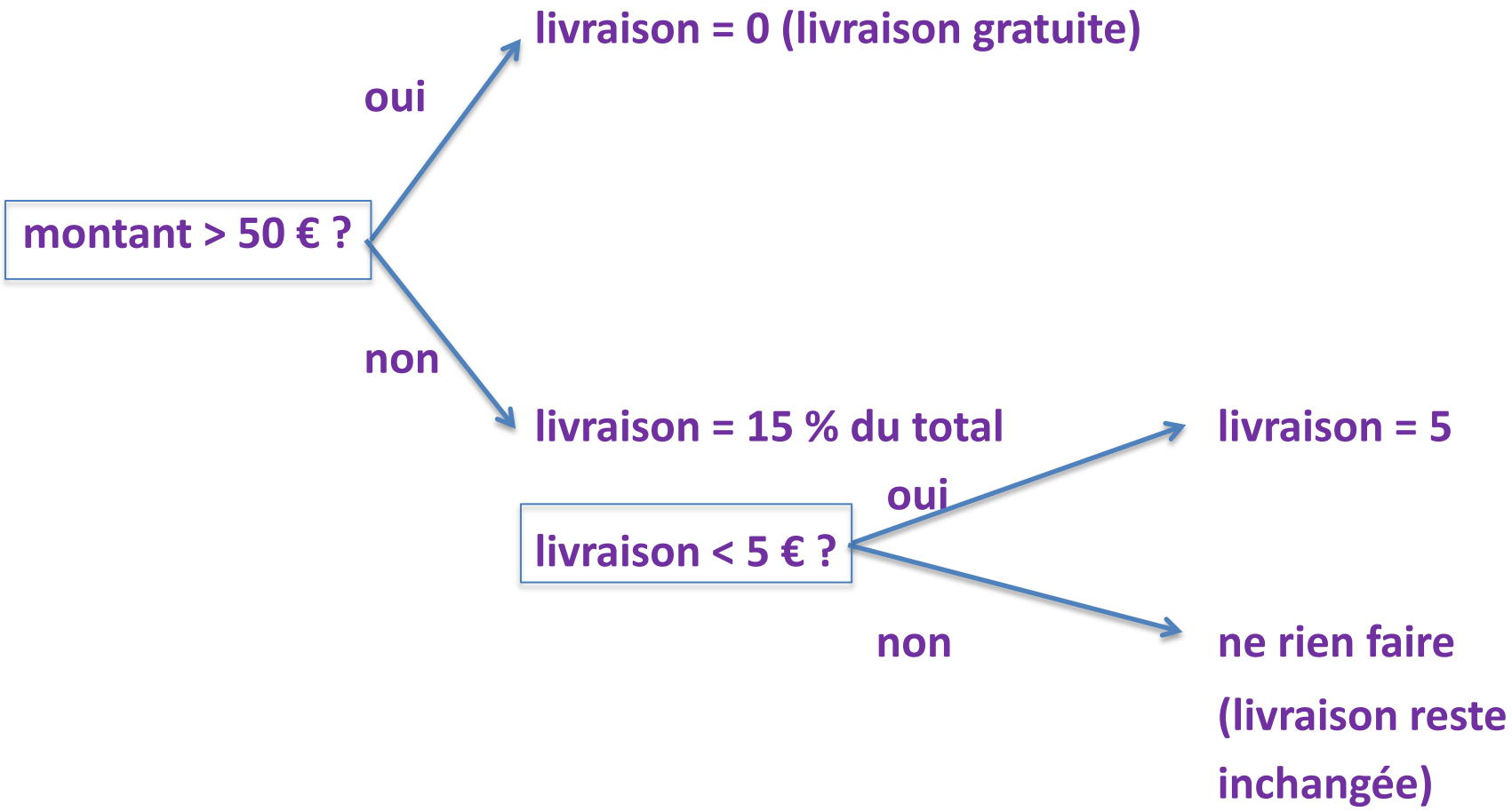
Mettre en place les tests dans notre exemple

```
Algorithme EauxMinerales
    …
 Début
    //Saisir le nom du client
    Afficher "Saisir le nom du client : "
    Saisir nomClient

    //Saisir le nombre de bouteilles
    Afficher "Quel est le nombre de bouteilles : "
    Saisir nbBouteilles

    //Calculs
    …
```

Rappel de l'arbre à mettre en place



Mettre en place les tests dans notre exemple

```
Algorithme EauxMinerales
    …
Début
    …
    //Calculs et tests
    montant = PRIX_UNITAIRE * nbBouteilles
    Si montant > MONTANT_MIN Alors
        livraison = 0
    Sinon
        livraison = montant * TAUX_LIVRAISON
        Si livraison < MIN_FORFAIT Alors
            livraison = MIN_FORFAIT
        FinSi
    FinSi
    total = montant + livraison
```

Version naïve

Mettre en place les tests dans notre exemple

```
Algorithme EauxMinerales
    …
Début                                          Version optimisée
    …
    //Calculs et tests
    montant = PRIX_UNITAIRE * nbBouteilles
    livraison = 0  //je pars du principe que la livraison est gratuite
    Si montant < MONTANT_MIN Alors
        livraison = montant * TAUX_LIVRAISON
        Si livraison < MIN_FORFAIT Alors
            livraison = MIN_FORFAIT
        FinSi
    FinSi
    total = montant + livraison
    …
```

Plan

1. Introduction
2. Structures alternatives dans un algorithme
3. Mettre en place les tests dans notre exemple
4. La structure Selon…Cas

Structures Selon…Cas

La structure Selon Cas est également une structure alternative qui sert dans le cas spécial où l'on teste uniquement l'égalité d'une variable avec un entier. Prenons l'exemple d'un menu :

```
Algorithme Menu
    Déclarations
        Variables
            choix : entier
Début
    Afficher "1-Saisir un nouveau client"
    Afficher "2-Afficher fiche client"
    Afficher "3-Modifier fiche client"
    Afficher "4-Supprimer un client"
    Afficher "5-Quitter"
    Afficher "→Choix"
    Saisir choix
```

```
Si choix == 1 Alors
    //…
Sinon
    Si choix == 2 Alors
        //…
    Sinon
        Si choix == 3 Alors
            //…
```

Structures Selon…Cas

La structure Selon Cas permet de traiter les actions selon la variable choix sur le même niveau sans indenter :

```
Algorithme Menu
    Déclarations
         Variables
            choix : entier
Début
    …
    Selon choix
       Cas 1 : //idem Si choix == 1 Alors
           //actions si choix == 1
       Cas 2 :
           //…
       Cas 3 :
           //…
       Cas 4
       Cas 5 :
           Afficher "Fin"
       Cas Sinon
           Afficher "Erreur choix"
    FinSelon
```